TP 2 : Types avancés et sécurité

Approche objet-relationnelle, utilisateurs et privilèges

Module : Bases de Données Avancées
Niveau : M1
Responsable du TP : M. Alachaher
Année universitaire : 2026/2027
Environnement : PostgreSQL / pgAdmin

Objectif

Découvrir les types de données avancés de PostgreSQL (types composites, tableaux et JSON simple), puis gérer les rôles et les droits d'accès. Chaque partie correspond à une séance de 1 h 30.

Préparation du dataset

Dans pgAdmin, créer une base de données nommée `tp2_avancees`, puis ouvrir son *Query Tool* et exécuter le fichier `dataset_tp2.sql`. Toutes les tables sont créées dans le schéma `public`.

Objet	Description
`departement`	Identifiant, nom, ville et budget.
`employe`	Identité, département, salaire, adresse composite, compétences (tableau) et préférences (JSON).
`projet`	Projets, budgets et statuts.
`affectation`	Association entre employés et projets, avec nombre d'heures.
`adresse_postale`	Type composite contenant rue, ville et code postal.

Le dataset contient 4 départements, 12 employés, 5 projets et 12 affectations.

Remarque

Le script réinitialise les quatre tables et le type composite du TP. Ne pas le relancer après avoir commencé les exercices, sauf pour repartir de zéro. Les rôles PostgreSQL sont des objets du serveur : ils ne sont pas supprimés par ce script.

Partie 1 — Bases de données objet-relationnelles

Objectif

Manipuler des données structurées dans PostgreSQL, sans utiliser de base documentaire externe : type composite, tableau et JSON.

Rappel syntaxe / Exemple

Un **type composite** regroupe plusieurs champs, par exemple les informations d'adresse d'un étudiant. Une colonne de type `TEXT[]` contient un tableau de textes, comme la liste des matières suivies par un étudiant. Le type `JSON` conserve une valeur JSON, et les opérateurs `->` et `->>` permettent d'en extraire respectivement une valeur JSON ou du texte.

```
-- Acces a la ville de residence (type composite) d'un etudiant
SELECT nom, (adresse).ville
FROM public.etudiant;

-- Recherche d'une matiere (type TEXT[]) suivie par un etudiant
```

```sql
SELECT nom, matieres
FROM public.etudiant
WHERE 'Bases de donnees' = ANY(matieres);

-- Extraction de la langue preferee (type JSON) d'un etudiant
SELECT nom, preferences ->> 'langue' AS langue
FROM public.etudiant;
```

`ARRAY['Mathematiques','Informatique']` construit un tableau de textes, tandis que `unnest(matieres)` transforme les elements du tableau en lignes. Pour acceder a un champ d'un type composite, comme la ville de residence, on utilise `(adresse).ville`.

Exercice 1 — Explorer les types structurés

1. Afficher le nom, le prénom et la ville de résidence de tous les employés à partir de l'attribut composite `adresse`.
2. Afficher les employés habitant à Oran, triés par nom.
3. Afficher les employés qui ont la compétences Design.
4. Afficher les employés qui ne sont pas d'Alger et qui préférant le télétravaille.

Exercice 2 — Utiliser le type JSON simple

1. Afficher le nom et le poste de chaque employé et sa langue préférée.
2. Modifier les préférences de l'employé 108 pour indiquer `{"langue":"fr","teletravail":true}` en utilisant une valeur de type `JSON`.
3. Ajouter pour l'employés 102 une préférence `'hybride' :'false'`.
4. Ajouter pour tous les employés qui préféré le teletravail une préférence `'hybride' :'true'`. fait le contraire pour les autre employes.

Exercice 3 — Manipuler les tableaux

1. Afficher le nom des employés et le nombre de compétences de chacun à l'aide de `cardinality()`.
2. Ajouter la compétence `Docker` à l'employé 104 avec `array_append()` ; vérifier le résultat.
3. Afficher une ligne par compétence pour l'employé 102 avec `unnest()`.
4. Trouver les employés possédant à la fois les compétences `SQL` et `Python` et il préférant le français.

Exercice 4 — Concevoir et analyser

1. Ajouter à `employe` une colonne `contacts` de type `TEXT[]` et renseigner deux contacts pour l'employé 101.
2. Écrire une requête affichant, pour les employés du département Informatique, le nom, la ville de résidence, les compétences et la langue préférée.
3. Crée un noveau type composite `infos_profesionnel` qui regroupe le poste, salaire et date d'embauche de l'employe.
4. Réflexion : pour quelles informations utiliseriez-vous un type composite, un tableau ou une table relationnelle séparée ?.

Partie 2 — Gestion des utilisateurs, privilèges et sécurité

Objectif

Définir des rôles adaptés aux métiers d'une entreprise, attribuer des droits précis sur les tables du dataset et vérifier les accès autorisés ou refusés.

Rappel syntaxe / Exemple

Un **rôle** représente un profil d'accès. `CREATE ROLE ... NOLOGIN` crée un rôle métier sans connexion directe. `GRANT` accorde des privilèges, `REVOKE` les retire. Il est possible de limiter `UPDATE` à une seule colonne.

```
-- Exemple de syntaxe (independant des exercices)
CREATE ROLE auditeur_interne NOLOGIN;
GRANT USAGE ON SCHEMA public TO auditeur_interne;
GRANT SELECT ON public.departement TO auditeur_interne;
GRANT SELECT (id_etudiant, nom, prenom) ON etudiant TO auditeur_interne;
REVOKE SELECT  ON etudiant FROM auditeur_interne;
-- Administrateur : autoriser le compte courant
-- SELECT current_role;
-- SET ROLE auditeur_interne;
-- RESET ROLE;
```

Le droit `USAGE` sur le schéma ne donne pas automatiquement accès aux tables. Le droit `SELECT` sur quelques colonnes ne permet pas d'utiliser `SELECT *` si d'autres colonnes sont interdites.

Remarque

Pour des tests fiables : utiliser un compte non superutilisateur lors de `SET ROLE`, vérifier qu'aucun privilège supplémentaire n'est accordé via `PUBLIC` ou un autre rôle, et revenir à `RESET ROLE` après chaque essai. Les `GRANT`/`REVOKE` doivent être effectués sous le compte administrateur. Les droits sur une colonne ne remplacent pas les droits nécessaires sur les autres tables éventuellement utilisées.

Exercice 1 — Définir les profils de l'entreprise

1. A l'aide de `SELECT` afficher `current_user` et `session_user`. Identifier les départements de l'entreprise.
2. Créer quatre rôles métier sans connexion directe : `responsable_rh`, `gestionnaire_finance`, `chef_projet` et `stagiaire`.
3. Accorder à ces quatre rôles le droit d'utiliser le schéma `public`.
4. En utilisant la commande `SET ROLE`, activer le rôle `responsable_rh`, puis vérifier le changement en affichant `current_user`.

Exercice 2 — Accorder les privilèges selon les métiers

1. **Responsable RH :** autoriser la consultation de `employe` et `departement`, puis la modification seulement des colonne `poste` et `date_embauche` de `employe`.
2. **Gestionnaire finance :** autoriser la lecture des colonnes `id_employe`, `nom`, `prenom`, `salaire` de `employe`, ainsi que la consultation de `departement` et `projet`. Autoriser la modification de `budget` dans `projet` uniquement.
3. **Chef de projet :** autoriser la consultation de `projet`, `affectation` et des seules colonnes `id_employe`, `nom`, `prenom` de `employe`. Autoriser la modification de `heures` dans `affectation`.

4. **Stagiaire :** autoriser uniquement la lecture de `departement` et `projet`. Ne lui accorder aucun privilège de modification.

Exercice 3 — Tester les autorisations et restrictions

1. Activer `responsable_rh`. Modifier le poste de l'employée 101, puis tenter de modifier son salaire. Comparer les deux résultats.
2. Activer `gestionnaire_finance`. Consulter les noms et salaires des employés, augmenter le budget du projet 202 de 10 000 DA, puis tenter de modifier le poste de l'employé 103.
3. Activer `chef_projet`. Modifier les heures de l'affectation (109, 201), puis tenter de consulter le salaire de l'employée 101. Pour chaque essai, préciser le privilège nécessaire ou manquant.
4. Activer `stagiaire`. Consulter les projets, puis tenter de consulter `employe` et de modifier le budget du projet 201. Noter les résultats. Revenir au rôle initial.

Exercice 4 — Révoquer des droits et analyser la sécurité

1. Retirer à `chef_projet` le droit `UPDATE` sur la colonne `heures` de `affectation`. Tester de nouveau une modification de ces heures.
2. Retirer à `stagiaire` le droit de consulter `projet`. Vérifier qu'il peut toujours consulter `departement`, mais plus `projet`.
3. Retirer au `gestionnaire_finance` le droit de consulter le `nom et prénom` de l'employé. Vérifier qui peut afficher le salaire mais pas le nom et prénom de l'employé.
4. Retirer au `responsable_rh` la modification de la `date_embauche` de l'employé. Tester les modifications effectuer.
5. Réflexion :Proposer une amélioration : comment limiter la consultation des employés par un responsable RH à certaines informations seulement, sans lui donner accès à toutes les colonnes ?

Bilan du TP

À la fin des deux séances, vous devez savoir : (1) interroger un type composite ; (2) manipuler des tableaux et des valeurs JSON ; (3) créer des rôles ; (4) attribuer, tester et retirer des privilèges.